CSE3500

In-Depth Analysis of the Rabin Karp Algorithm

Charlotte Manalo, Jason Pondugula, Nina Cunha, Arnav Mahajan

Nina: I began report with the main goal of this report, as explained in the introduction. I introduced Naive String-Matching Algorithm to establish a foundation of the Rabin-Karp Algorithm. After that I explained the basic mechanism of how a rolling hash works with an example because it is the base of the entire Rabin-Karp algorithm. I built up to the specific implementation used in the Rabin-Karp Algorithm. Next, I mention how Rabin-Karp would be helpful in a real-world example. In addition to writing the report, I took the lead in organizing the presentation slides, ensuring they were visually engaging and effectively conveyed the core concepts and findings. I organized the content into clear sections that complemented the report's structure, focusing on key points to enhance understanding during the presentation. I also ensured that our works cited were in the correct formatting along with noting any AI usage in the proper format. I then reviewed and edited any grammatical or formatting issues while keeping the integrity of my other groupmates work. Alongside this, I consistently kept the team informed about upcoming deadlines and project requirements throughout the process. This involved setting reminders, coordinating tasks, and maintaining open communication to ensure that everyone stayed on track and aligned with the project's goals.

Charlotte: I was responsible for researching and writing the *Implementation Details* and *Results and Analysis* sections of the project, focusing specifically on explaining how Rabin-Karp's rolling hash works in code. I broke down the algorithm step-by-step, starting with the initial setup (defining base, modulus, and lengths of the text and pattern) and explained how the rolling hash helper `h` is precomputed to allow efficient updates. I then detailed how the initial hashes for the pattern and the first text window are calculated and described the sliding window process, including when and how hash values are compared and updated. To help readers understand the process visually, I walked through an example of searching for "ABAC" in "ABAAABCDABACDABACABAB," explaining the expected outputs. I made sure that every line of code had a clear, concise explanation to ensure that readers could easily follow how the algorithm operates. In addition to writing these sections, I collaborated with the team by regularly communicating updates in the group chat, making sure the technical sections stayed on schedule alongside the rest of the report.

Jason: I started my work by writing the **comparisons to other algorithms** section, where I thoroughly researched and explained the strengths and weaknesses of **Boyer-Moore** and **KMP** in relation to **Rabin-Karp**. This section focused on highlighting the key differences in how these algorithms function and their performance across different scenarios. I also researched and wrote detailed notes on both comparison algorithms to ensure a clear understanding of their mechanisms and added them to our group folder. Following that, I took charge for writing the **justification** section, where I explained why Rabin-Karp is the best choice for certain applications, emphasizing its strengths, particularly in handling multiple patterns simultaneously. I also took the initiative to **double-check and proofread** the **introduction**, ensuring the clarity and coherence of the content. To visually demonstrate the time complexity differences between the algorithms and add visuals to our report, I created a **graph comparing their time**

complexities, which helped provide a clear comparison for readers as well. Throughout the project, I made it a point to **regularly check in** with everyone in the group via our group chat, ensuring communication remained open. Although Nina created the majority of the documents, I played an active role in **helping create documents**, assigning tasks, and ensuring that everyone stayed on track by enforcing task assignments and deadlines in our groupchat especially.

Arnav: I contributed to the project by writing the Time Complexity and Formal Proof section for the Rabin-Karp algorithm. I started by establishing the recurrence relation T(n), explaining how the algorithm operates in O(n+m) time in the best and average cases due to efficient hash computations and rolling updates. I also explained how, in the worst case, the algorithm can degrade to O(nm) time when frequent hash collisions occur, making it equivalent to brute-force string matching. To ensure clarity, I carefully broke down the complexity into phases — initial hash computation, rolling hash updates, and full string comparisons — and connected the analysis directly to the algorithm's practical performance. I also refined the introduction that was initially written by Nina. We all maintained great communication with each other, letting each other know what we needed to do and when we would have it done.

Charlotte Manalo

Jason Pondugula

Nina Cunha

Arnav Mahajan

Rabin Karp Algorithm

Introduction:

This report provides a complete analysis of the Rabin-Karp Algorithm. From its wide usage in plagiarism and gene detection, there is much to unpack. This is done via an in-depth explanation of its time and space complexity and evaluates its performance compared to alternative string-matching algorithms. It will explain why, in specific conditions, this algorithm is the most effective. Later, the thorough explanation of the implementation of the Rabin-Karp Algorithm will reinforce how this algorithm works on the code level, rounding out the knowledge of this very strong algorithm.

How Rabin-Karp Works:

Rabin-Karp is a string/pattern matching algorithm that works by hashing the pattern we are looking for and comparing it with the hash pattern in the larger string of characters we are searching in. It builds off the Naive String-Matching Algorithm, improving the time and space complexity.

Rabin-Karp gets the majority of its efficiency from the rolling hash. A rolling hash is computed by assigning a value to each character, and the sum of the characters is the hash for that given value. For example, if we are searching for the pattern “EFG” in a given string of characters. For simplicity, let's say that “EFG” hashes to the number 18, since that is the addition of the characters indices in the alphabet. The rolling hash allows us to adjust the hash of the characters we are looking at without having to rehash each time. We just subtract the character we are not using and add the next character in the sequence. This is how the Rabin-Karp algorithm computes pattern matching efficiently.

Pattern: "EFG" || String: "IDEEFG"

IDEEFG

Hash(IDE) = Hash (8 + 4 + 5) = 18 = Hash(Pattern)

- Check each value to determine a match
  - “E” ≠ “I”

- Move onto the next character

I**DEE**FG

Hash(DEE) = Hash(18 – 8 + 4) = 14 ≠ 18 *# 18 is the value of the previous hash, we are removing "I", and adding in "E"*

- Move onto the next character

ID**EEF**G

Hash(EEF) = Hash(14 – 4 + 6)_ = 16 ≠ 18

- Move onto the next character

IDE**EFG**

Hash(EFG) = 18 = Hash(Pattern)

- Check each value to determine a match
  - "E" = "E" ✓
  - "F" = "F" ✓
  - "G" = "G" ✓
- **Match at index: 3**

It will continue until the end of the string. Once the algorithm has checked the entirety of the string, the output of Rabin-Karp will be the starting indices where there was a pattern match. In this case where there are 2 instances of the pattern, it will output 3.

That is the basic functionality of how a rolling hash works. However, using a simple hash can lead to a lot of **hash collisions**, which can be seen by the example above. The first three characters had the same hash as the target pattern but were not the intended group of characters. This means that the algorithm must go through the extra steps of checking the individual characters in the pattern. This functionality, if it must be computed often, is essentially the exact algorithm of Naive String-Matching, which does not have good running time.

For the Rabin-Karp Algorithm, it implements a much more sophisticated rolling hash to eliminate hash collisions as much as possible, improving the running time. The rolling hash begins by choosing two values, called "*b"* and *"p",* where "b" is the base and "p" is the modulus of the hash function. The base is typically a prime number and also sometimes the size of the set of characters chosen (e.g. 26 for the alphabet and 256 for ASCII characters). *P* is also a prime number, and this is to ensure that the hash values have a good distribution among the hash 'buckets' they get assigned to. The general form for the Rabin Karp Hash is:

$$H = (c_1 \cdot b^{n-1}) + (c_2 \cdot b^{n-2}) + \ldots + (c_n \cdot b^0)$$

Where *n* is the length of the pattern being hashed. Rabin Karp is especially good for detecting plagiarism. This is due to how good it is at multiple pattern search. Let's say we are searching for a sentence in a sequence of characters to see if it was plagiarized off another person's work. The algorithm works by hashing each word in the target sentence. As it scans through the essay, it compares the hash of the current section of the paragraph to the hashes of the individual words. Since comparing numbers can be done in constant time (O(1)), Rabin-Karp is more efficient than other string-matching algorithms. The example I provided aligns directly with the implementation of the algorithm in our code. For a more detailed understanding, the code demonstrates how this example was applied effectively. [13][14]

Implementation Details

Results and Analysis: *Explain in code how to calculate rolling harsh (show how it happens)*

On the left is the basic implementation of The Rabin-Karp Algorithm.

- **Initial Setup**
  - base = 256: Represents the number of possible ASCII characters.
  - mod = 101: A prime number used to keep hash values manageable.
  - n **and** m: Store lengths of the text and pattern, respectively.
- **Precompute** h **(Rolling Hash Helper)**
  - h = 1 → h = (h * base) % mod: Computes base^(m-1) % mod to help remove the leftmost character efficiently.
- **Hash Calculation for Pattern & First Text Window**
  - pattern_hash: Computed by iterating over the pattern.
  - text_hash: Computed for the first m characters of the text.
- **Sliding Window & Hash Comparison**
  - for i in range(n - m + 1): Iterates through all possible starting positions of the pattern in the text.
  - if pattern_hash == text_hash: If hashes match, checks for an exact string match.
  - **Rolling Hash Update**:

- Removes the contribution of the outgoing character (text[i]).
- Adds the contribution of the incoming character (text[i + m]).
- Adjusts for negative hash values (if text_hash < 0).

- **Example Usage**
  - Searches for "ABAC" in "ABAAABCDABACDABACABAB" and prints matching indices.

Output:

Pattern found at index 8
Pattern found at index 14
Pattern found at index 18

```
def rabin_karp(text, pattern):
    # Set base and modulus for the rolling hash function
    base = 256  # Number of possible characters (extended ASCII)
    mod = 101   # A prime number (modulus for hash function)

    # Lengths of text and pattern
    n = len(text)
    m = len(pattern)

    # Calculate hash values for the pattern and the initial window of the text
    pattern_hash = 0
    text_hash = 0
    h = 1  # The value of base^(m-1) % mod

    # Precompute h = base^(m-1) % mod
    for i in range(m - 1):
        h = (h * base) % mod

    # Compute the hash value of the pattern and the first window of text
    for i in range(m):
        pattern_hash = (base * pattern_hash + ord(pattern[i])) % mod
        text_hash = (base * text_hash + ord(text[i])) % mod

    # Slide the pattern over text one by one
    for i in range(n - m + 1):
        # If the hash values match, check for exact match
        if pattern_hash == text_hash:
            # Check the actual characters if hash matches
            if text[i:i + m] == pattern:
                print(f"Pattern found at index {i}")

        # Calculate the hash value for the next window of text
        if i < n - m:
            text_hash = (base * (text_hash - ord(text[i]) * h) + ord(text[i + m])) % mod

            # We might get negative value of text_hash, convert it to positive
            if text_hash < 0:
                text_hash += mod

# Example usage:
text = "ABAAABCDABACDABACABAB"
pattern = "ABAC"
rabin_karp(text, pattern)
```

Time Complexity/Proof:

Variables and Initialization:

- **n** = length of the text
- **m** = length of the pattern
- **p** = 256 (number of characters in character set)
- **mod** = a large prime number (to avoid overflow and collisions)
- **hpattern** = 0 (hash value for the pattern)
- **htext** = 0 (hash value for current window of text)
- **h** = 1 (value of pm−1mod modp^{m-1} \mod \text{mod}pm−1modmod)

Preprocessing:

Compute h=pm−1mod modh = p^{m-1} \mod \text{mod}h=pm−1modmod:

```
for i in range(m - 1):
   h = (h * p) % mod
```

Compute initial hashes for pattern and first text window:

```
for i in range(m):
   hpattern = (p * hpattern + ord(pattern[i])) % mod
   htext = (p * htext + ord(text[i])) % mod
```

Pattern Matching

```
for i in range(n - m + 1):
   if hpattern == htext:
      if text[i:i + m] == pattern:
         print("Pattern found at index", i)
```

If not at the end, update hash for next window:

```
if i < n - m:
    htext = (p * (htext - ord(text[i]) * h) + ord(text[i + m])) % mod
    if htext < 0:
        htext += mod
```

Best and Average Case Complexity

- **Initial Hash Computation:**
  - Computing the hash values for the pattern and the first window of text takes **O(m)** time.
- **Rolling Hash Updates:**
  - For each subsequent window in the text, the hash can be updated in O**(1)** time using the rolling hash formula.
- **Expected Comparisons:**
  - If hash collisions are rare (as expected with a good hash function and a large prime modulus), comparing the actual strings only occurs occasionally, with an **expected cost of O(1)** per match.
- **Total Expected Time:**

  $T(n)=O(m)+O(n)$

  $T(n) = O(n+m)$

This includes O(m) for initial hash computation and O(n) for scanning through all windows in the text.

Worst-Case Complexity

- In the worst case, **hash collisions occur at every step**, forcing a full character-by-character comparison between the pattern and each substring in the text.
- Each such comparison takes **O(m)** time.
- **Total Worst-Case Time:**

  $T(n)=O(m)+n\times O(m)$

  $Tn = O(n\cdot m)$

Naive String Matching:

The Naive String-Matching algorithm works by taking in a string pattern, *"n"* that will comprise of the pattern, and a larger string, *"s"*, in which we are searching for the pattern in. It starts by comparing the first value of *n* with the first value of *s,* if there is not a match it moves over one character and repeats until all of *s* has been parsed through.

The output will be the index of the pattern match. However, even with a small string of characters, the naive algorithm computes a lot of comparisons to determine if a pattern has been found. Best case time complexity for the Naive method is O(n), where it only has to search the length of the pattern. In the worst case, especially when the pattern and the text are similar (e.g., both are repeated characters like "aaaaa"), the algorithm checks each possible starting position in the text and compares up to the full length of the pattern each time. This leads to a time complexity of $O(n^2)$.

Best Case	O(n)
Worst Case	$O(n^2)$

Boyer-Moore String Search:

Boyer-Moore is another string searching algorithm that scans the pattern from right to left in the text. Before searching starts, the algorithm preprocesses the pattern to build two tables: the bad character table and the good suffix table, which allows Boyer-Moore to skip over sections of the text that cannot possibly match the pattern. The bad character rule means that when a mismatch occurs, the pattern is shifted such that the mismatched character in the text aligns with the rightmost occurrence of that character in the pattern, or it shifts past the mismatch if the character is absent from the pattern. The good suffix rule means that if a mismatch happens after a partial match of the pattern, the pattern is shifted based on the longest suffix of the pattern that is also a prefix or appears elsewhere in the pattern, thus reducing the number of comparisons. The functional difference between this algorithm and Rabin-Karp lies in their approach to pattern matching: Boyer-Moore uses **heuristics** (bad character and good suffix rules) to skip sections of the text, performing comparisons from right to left, while Rabin-Karp relies on **hashing** to compute the hash values of the pattern and text substrings for quick comparisons. Boyer-Moore focuses on reducing comparisons through preprocessing, while Rabin-Karp focuses on hashing to efficiently check potential matches.

The functional difference between Boyer-Moore and Rabin-Karp lies in how they approach pattern matching: Boyer-Moore uses two heuristics, the bad character rule and the good suffix rule, to skip sections of the text that cannot possibly match the pattern, performing comparisons from right to left. The bad character rule shifts the pattern to align the mismatched character with its rightmost occurrence in the pattern or skips entirely if the character is not

present. The good suffix rule shifts the pattern based on the longest suffix that matches a prefix or occurs elsewhere in the pattern, minimizing unnecessary comparisons. In contrast, Rabin-Karp employs a hashing approach, where it computes the hash values of the pattern and substrings of the text and compares these hash values. If the hashes match, a full comparison is performed to confirm the match, allowing it to efficiently check multiple patterns simultaneously by comparing the hashes of all patterns in one pass. While Boyer-Moore minimizes comparisons using preprocessing heuristics, Rabin-Karp relies on hashing to quickly eliminate non-matching substrings.

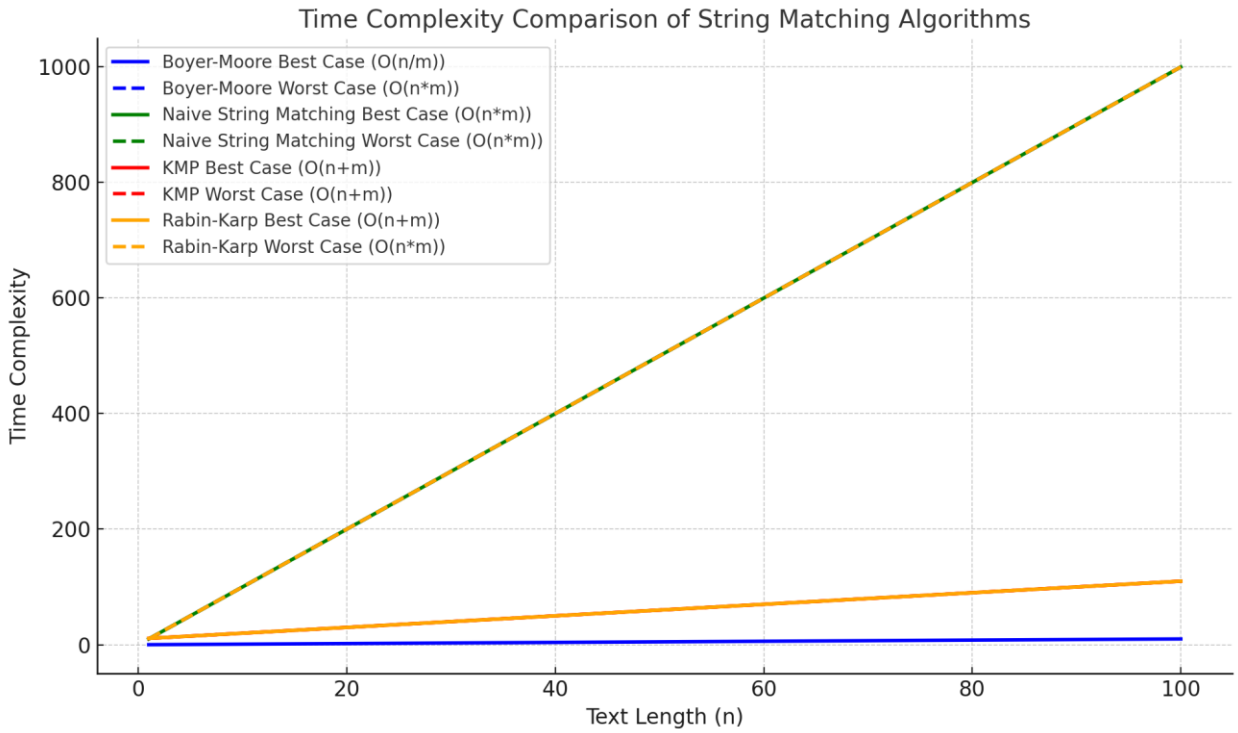
Best Case	O(n/m) where n is length of text, m is length of pattern
Worst Case	O(n * m)

Knuth-Mooris-Pratt (KMP) String Matching:

Knuth-Morris-Pratt (KMP) is another string searching algorithm that scans the pattern from left to right in the text. Before searching starts, the algorithm preprocesses the pattern to build the Longest Prefix-Suffix (LPS) table, which allows KMP to avoid unnecessary comparisons by using information about previously matched portions of the pattern. The LPS table stores the length of the longest proper prefix of the pattern that is also a suffix. This table helps determine how far the pattern can be shifted when a mismatch occurs, ensuring that previously matched characters are not checked again.

The functional difference between KMP and Rabin-Karp lies in their pattern matching strategies: KMP uses the LPS table (Longest Prefix-Suffix table) to determine how much to shift the pattern after a mismatch. This table allows KMP to avoid revisiting portions of the pattern that have already been matched, optimizing the search by using previously matched prefixes to guide the shifts. When a mismatch occurs, KMP shifts the pattern by an amount determined by the LPS table, skipping over portions that have already been compared. On the other hand, Rabin-Karp utilizes hashing to compute the hash values of the pattern and substrings in the text. It compares the hash values to identify potential matches quickly and performs full comparisons only when the hashes match. While KMP minimizes redundant comparisons through preprocessing the pattern's structure, Rabin-Karp accelerates matching by leveraging hashing for efficient substring comparison across multiple patterns.

Best Case	O(n + m) where n is length of text, m is length of pattern
Worst Case	O(n + m)



Justification:

The Rabin-Karp algorithm is particularly strong in scenarios where multiple patterns need to be searched for simultaneously. Unlike Boyer-Moore and KMP, which are optimized for single-pattern searches, Rabin-Karp can handle multiple patterns efficiently in one single pass. By using hashing to compute hash values for all patterns and comparing them against substrings in the text, Rabin-Karp eliminates the need for multiple individual searches. Boyer-Moore, while very efficient for single-pattern matching, requires multiple passes for each pattern, resulting in increased overhead when searching for several patterns. KMP also focuses on single-pattern searches and performs well by preprocessing the pattern to build the LPS (Longest Prefix-Suffix) table, but it too would need to conduct separate searches for each pattern. In contrast, Rabin-Karp's hashing allows it to perform one search for all patterns, making it ideal for applications like plagiarism detection or search engines that need to search through multiple keywords at once.

Another key strength of Rabin-Karp lies in its simplicity and flexibility. The algorithm is straightforward to implement, especially with its hashing-based approach. This simplicity allows for easy adaptations to handle wildcards, regular expressions, or even approximate matching scenarios, which is more complex with Boyer-Moore and KMP. Boyer-Moore requires complex preprocessing to build the bad character and good suffix tables, which, while efficient, are more difficult to modify for tasks like approximate matching or multiple pattern searches. Similarly, KMP relies on the LPS table for pattern shifts, which, while efficient for matching single patterns, makes it harder to extend for approximate matching or for handling multiple patterns. From a time complexity perspective, Rabin-Karp offers a best-case time complexity of O(n + m), where n is the length of the text and m is the length of the pattern. Rabin-Karp's flexibility makes it useful in various fields like for bioinformatics tasks, where slight variations between the pattern and the text are common, such as when searching DNA sequences that may have mutations.

Rabin-Karp also has significant advantages when dealing with large datasets containing repeated patterns. In such cases, the hashing mechanism in Rabin-Karp allows it to efficiently compare the hashes of substrings with the pattern's hash. If the hashes match, the algorithm performs a full comparison to confirm the match. In contrast, Naive String Matching performs a direct comparison for every substring, which results in a time complexity of O(n * m) in the worst case, making it much slower for large datasets. While Boyer-Moore and KMP are more efficient than the naive approach for single-pattern searches, they are not optimized for handling multiple patterns in one pass. Rabin-Karp, by leveraging hashing, speeds up the process of pattern matching significantly compared to the naive approach, especially in cases where multiple patterns are involved or when there are many repeated substrings.

References

[1] A. Mitchell, "The Rabin-Karp algorithm explained," Expertbeacon, https://expertbeacon.com/the-rabin-karp-algorithm-explained/ (accessed Apr. 15, 2025).

[2] "Boyer Moore algorithm for Pattern Searching," GeeksforGeeks, https://www.geeksforgeeks.org/boyer-moore-algorithm-for-pattern-searching/ (accessed Apr. 15, 2025).

[3] P. Dave, "Naiive, Knuth-Mooris-Pratt and Rabbin Karp string matching algorithms," Medium, https://blog.devgenius.io/naiive-knuth-mooris-pratt-and-rabbin-karp-string-matching-algorithms-a54cb141dd7a (accessed Apr. 15, 2025).

[4] V. Joshi, "(PDF) report and analysis of Rabin Karp algorithm," Report And Analysis Of Rabin Karp Algorithm,

https://www.researchgate.net/publication/381002142_Report_And_Analysis_Of_Rabin_Karp_Algorithm (accessed Mar. 27, 2025).

[5] C. Jayamina, "Compare contrast Rabin Karp algorithm vs Knuth-Morris-Pratt algorithm," Medium, https://chandimajayamina.medium.com/compare-contrast-rabin-karp-algorithm-vs-knuth-morris-pratt-algorithm-9ac8de452b74 (accessed Mar. 27, 2025).

[6] M. Afzalpasha, "Rabin-Karp Algorithm: A tool for string matching," Medium, https://medium.com/@md.afzalpasha10/rabin-karp-algorithm-a-powerful-tool-for-string-matching-f1104e3b1107 (accessed Mar. 27, 2025).

[7] "Rabin-Karp algorithm for pattern searching," GeeksforGeeks, https://www.geeksforgeeks.org/rabin-karp-algorithm-for-pattern-searching/ (accessed Mar. 13, 2025).

[8] "Rabin-Karp algorithm," Programiz, https://www.programiz.com/dsa/rabin-karp-algorithm (accessed Mar. 13, 2025).

AI Tool Usage

[9] Nina Copilot, "Generated ideas for algorithms that we could use," 2/20/2025.

[10] Nina ChatGPT (GPT-4o-mini), "Asked for websites that explain the real-world use cases of the Rabin-Karp Algorithm," 3/10/2025.

[11] Nina ChatGPT (GPT-4o-mini), "Generated websites to learn more about the rolling hash; asked for a step-by-step example for better understanding," 3/26/2025.

[12] Charlotte Manalo ChatGPT (GPT-4o), "Asked to generate algorithms to compare to Rabin-Karp," 3/27/2025.

[13] Nina Copilot, "Asked how I can improve my explanation of the algorithm," 4/19/2025.

[14] Nina Copilot, "Asked how I can improve readability of the final paragraph explaining the Rabin-Karp algorithm," 4/20/2025.

[15] Arnav ChatGPT (GPT-4o), "Formatted my information to make it clearer and more concise," 4/24/2025.